POS financial services review and corrections

Execution resumed 2026-09-24 against the preserved 2026-09-23 run baseline. Scope: `services/pos/pos.service.ts`, its service tests, checkout lifecycle/commit recovery contracts, and disposable PostgreSQL verification. Existing dirty-worktree implementations and the prior checkout UI changes are preserved. This report records source and bounded test evidence; it is not a financial, security or release certification.

Corrected findings

ID	Severity / actor / journey	Reproduction and impact	Correction / source	Verification
POS-SVC-01	High; cashier/support; sale completion after commit	Fail the completion registry update, fail its follow-up read, or return no completed record after the financial transaction committed. Previously the operation could reject even though stock/payment/ledger and its COMMITTED envelope already existed, encouraging another attempted sale.	`pos.service.ts:3023-3059`: completion hydration failures now return the durable transaction result with `receiptStatus: RETRY_REQUIRED`; a completed racing result is still validated. No financial transaction is repeated.	Red proof `services-completion-red.log`; three completion failure regressions in `pos.service.test.ts:1160`; final 109-test run passes. These are mock fault-injection tests, not network-failure proof against a real payment provider.
POS-SVC-02	High; finance/refund operator; electronic refund/void	Pass a completed CARD, MOBILE_MONEY, BANK_TRANSFER or STORE_CREDIT sale to correction services. The old service could mark it refunded/cancelled and post correction effects without authoritative provider reversal. MIXED also lacked an explicit unsupported-rail result.	`pos.service.ts:3133`: shared guard accepts CASH and CREDIT only; refund separately rejects CREDIT, leaving cash refund and cash/on-account void as supported service paths. Returns a safe, explicit unsupported-payment-rail error before monetary/stock mutations.	Red proof `services-correction-red.log`; parameterized unsupported-rail regressions `pos.service.test.ts:1281`; final run passes. No provider reversal was invented or invoked.
POS-SVC-03	High; cashier/finance; refund/void drawer integrity	Configure an older drawer at the same terminal and a different drawer referenced by the shift's OPENING_BALANCE. The old outflow selected the oldest drawer and performed an unconditional absolute-balance update. That could deduct from the wrong drawer or overwrite a competing balance change.	`pos.service.ts:3309`: outflow requires exactly one scoped opening-drawer record, an open drawer whose balances equal the authoritative shift snapshot, sufficient funds, and a successful compare-and-swap. A lost claim aborts the transaction before refund/payment posting. `requireActiveCorrectionSession` also verifies the terminal is active and still linked to this shift.	Initial failed tests `services-drawer-red.log` (includes a corrected empty-array test-data shape issue); final tests cover lost claim, absent/duplicate/mismatched opening evidence, displaced terminal and insufficient funds. Real PostgreSQL `services-postgres-corrections-final.log`: four tests prove correct drawer selection for refund and void, one concurrent refund outflow, and rollback after a later posting failure.

The drawer change stays within the existing service contract: no schema change, public API change, new payment method or migration. Existing rows that lack consistent opening evidence now stop with an explicit

conflict instead of silently choosing an unrelated drawer. Repair requires investigating the shift and drawer evidence, not creating a fabricated opening record.

Services and controls traced

Graph discovery used the action graph and the architecture review's `pos_tender_actions_ts` community 133. Current source, rather than historical graph line positions, established these dependencies.

Journey	Authoritative service path / prerequisites	Outcome, failure and evidence
Shift open	`openPOSShift`; tenant/location/terminal/cashier scope, writable module admission, active terminal claim	Creates the shift/opening drawer evidence and updates terminal ownership in one transaction. Unit coverage includes a lost terminal claim.
Cart reads and edits	`getOrCreateDraftCart`, `getActivePOSCart`, `addPOSCartLine`, `updatePOSCartLine`, `removePOSCartLine`; draft owner, active linked shift, writable cart access and checkout lease	Server item price/tax/quantity calculations; serializable edit transactions retry proven P2034 aborts only. Unknown transport outcomes are not blindly retried.
Cart resume/park	`checkout.service.ts` and `checkout-state.ts`; owned lease, active shift, scoped cart and unresolved/offline-work checks	Resume revalidates current stock/prices and fences revisions; parking preserves draft lines; pending uncertain requests cannot be discarded through parking.
Cash/split tender	`assertPOSSaleMath` and `allocateTenders`; authoritative cart revision, consistent server totals and supported tender evidence	Decimal calculations and cash change allocation. UI availability remains CASH; multiple backend payment enums are not evidence of a delivered cashier payment journey.
Electronic sale support	`commitPOSSale` with provider-authority claims and payment evidence	A browser-supplied reference alone cannot prove capture. Mock service regressions cover manual-reference denial and claimed provider evidence. Live provider behavior and reversal integration are unverified here.
Cash/on-account sale	`commitPOSSale` → inventory stock issue → sale/payment ledger posting → customer receivable/ledger as applicable → event/outbox → COMMITTED result	One serializable financial transaction, plus guarded scoped result replay. The committed envelope records financial truth; receipt hydration is separate. Unit tests verify transaction client wiring and downstream failure behavior; see real-database scope limits below.
Recovery/checking	`getPOSSaleCommitStatus`, `readPOSSaleCommitStatusInTx`	Scoped hash-validated status reads; NOT_FOUND is uncertain, never proof of rollback. Retired cart plus closed shift can establish NOT_COMMITTED. Read-only recovery tests exclude write delegates; lifecycle database tests preserve original retained request evidence.
Refund/void	`refundPOSSale`, `voidPOSSale`; protected action/fresh-auth admission is owned by security review, service rechecks writable module access and own active shift	Full cash refund or cash/on-account void only. Sale state claim, session totals, correct drawer, payments, stock, posting and evidence are transaction-scoped. No cashier refund/void UI was added. Unsupported electronic corrections explicitly stop.
Shift close	`closePOSShift`; active session and unique opening evidence, offline/pending-work barriers, counted balance and variance explanation	Serializable close with exact replay/conflict semantics. Real database tests cover simultaneous close, competing active writer, and rollback on event/event-apply/outbox/audit failure.
Fiscal/receipt work	Commit event includes fiscalization outbox and nonauthoritative PENDING metadata; receipt generation after durable sale	A fiscal adapter or receipt failure does not erase recorded sale truth. No statutory rules changed; queued fiscalization is not statutory completion.

Accounting, stock valuation, customer credit and period-close behavior remain delegated to their existing kernels. Passing POS wiring tests does not independently certify those kernels. No direct POS purchasing/AP or payroll mutation was introduced; those domains are relevant only through verified upstream stock/identity and downstream finance reporting dependencies.

Verification and environment

Head revision: see `revision.txt`; source baseline: `source-baseline.json` and `baseline/services/pos/`. Final hashes and command metadata are in `services-verification.json`. The source tree is dirty, so HEAD alone does not identify the tested contents.

All Jest invocations use `TS_NODE_TRANSPILE_ONLY=true`, `TS_NODE_COMPILER_OPTIONS={"moduleResolution":"node"}`, `--runInBand` and focused POS test roots. These flags avoid compiling the entire TypeScript workspace while loading Jest configuration; the main task separately runs the project typecheck.

Check	Result	Evidence / actual scope
Focused service regression	PASS, 6 suites / 109 tests, exit 0	`services-regression-final.log`: pos.service, pos-shift-close.service, pos-cart-transaction, pos-customer.service, checkout-lifecycle.service, commit-recovery.service. Earlier 129-test run covered a different six-suite set; counts are not additive.
Changed service/test ESLint	PASS, exit 0	`services-eslint-final.log` (empty successful output).
Real PostgreSQL commit registry	PASS, 3 tests, exit 0, previous phase	`services-postgres-commit.log`: unique concurrent claim, cross-tenant source constraint, immutable lifecycle result. Tests exercise registry constraints, not a complete sale.
Real PostgreSQL shift close	PASS, 7 tests, exit 0	`services-postgres-close-final.log`, `services-postgres-exits-close.json`. Earlier two failures were an obsolete expected error string; tests now check the preserved safe error code/status/exposure and retain every rollback assertion.
Real PostgreSQL checkout lifecycle	PASS, 5 tests, exit 0	`services-postgres-lifecycle-final.log`, `services-postgres-exits-lifecycle.json`: competing owners, retained recovery through close, rollback of retirement/release, cross-tenant/cashier denial, late edit versus close. Uses limited cloned tables and enum types without full production constraints; scope is lifecycle transactions.
Real PostgreSQL financial corrections	PASS, 4 tests, exit 0	`services-postgres-corrections-final.log`, `services-postgres-exits-corrections.json`: full public schema, real POS transactions, drawer/payment/sale/shift rows. Stock, ledger, entitlement and business-event collaborators are explicitly simulated, and the test item is untracked. This establishes the tested database transaction boundaries, not full stock/ledger/RBAC integration.
Real PostgreSQL full commit	PASS, 2 tests, exit 0	`services-postgres-full-commit-final.log`, `services-postgres-exits-full-commit.json`: real inventory, accounting, customer/receivable, checkout and event kernels, with no internal mocks. Proves one financial result under concurrent duplicate completion and complete rollback at the final registry boundary followed by safe same-request retry. Synthetic zero-tax accounting rules; no external provider or statutory certification.

Check	Result	Evidence / actual scope
Full-commit test ESLint	PASS, exit 0	`services-full-commit-eslint-final.log` (empty).
Scoped diff whitespace check	PASS, exit 0	`git diff --check --` the five service/test files; Git emitted only existing CRLF normalization warnings.

The PostgreSQL total is **21 distinct test cases** with the different scopes described above:

1. Commit registry: exactly one concurrent unique claim.
2. Commit registry: cross-tenant source combination denied by the database constraint.
3. Commit registry: CLAIMED → COMMITTED → COMPLETED transition and final result immutability.
4. Shift close: one simultaneous close and one exact replay.
5. Shift close: one changed close and a durable idempotency conflict.
6. Shift close: ordering against an active cash writer.
7. Shift close: rollback on event insertion failure.
8. Shift close: rollback on event-application failure.
9. Shift close: rollback on outbox insertion failure.
10. Shift close: rollback on audit insertion failure.
11. Lifecycle: one concurrent owner and rejection of the losing tab's writes.
12. Lifecycle: retained request through closure and proof that its retired cart cannot submit later.
13. Lifecycle: rollback of retirement, ownership release and shift status together.
14. Lifecycle: foreign tenant/cashier denied despite known identifiers.
15. Lifecycle: cart edit versus shift closure, with late edits rejected.
16. Correction: cash refund uses the opening drawer and preserves the older drawer.
17. Correction: cash void uses the opening drawer and preserves the older drawer.
18. Correction: one concurrent refund, one outflow and one refund row.
19. Correction: posting failure rolls back sale, payment, drawer, shift and refund rows.
20. Full commit: concurrent duplicate requests and subsequent replay preserve one real stock/cash/customer/receivable/ledger/event result.
21. Full commit: proven late registry-finalization failure rolls back every real financial kernel, retains the request, and permits a successful same-request retry.

Commands (all paths relative to the workspace root):

```
$env:TS_NODE_TRANSPILE_ONLY='true'
$env:TS_NODE_COMPILER_OPTIONS='{"moduleResolution":"node"}'
node node_modules/jest/bin/jest.js --runInBand --roots services/pos/__tests__ --runTestsByPath
services/pos/__tests__/pos.service.test.ts services/pos/__tests__/pos-shift-close.service.test.ts
services/pos/__tests__/pos-cart-transaction.test.ts services/pos/__tests__/pos-
customer.service.test.ts services/pos/__tests__/checkout-lifecycle.service.test.ts
services/pos/__tests__/commit-recovery.service.test.ts --silent
node node_modules/eslint/bin/eslint.js services/pos/pos.service.ts
services/pos/__tests__/pos.service.test.ts services/pos/__tests__/pos-shift-close.postgres.test.ts
services/pos/__tests__/pos-commit-result.postgres.test.ts services/pos/__tests__/pos-
corrections.postgres.test.ts
node what-next/pos-enterprise-modernization/2026-09-23/evidence/run-pos-postgres-existing.cjs close
node what-next/pos-enterprise-modernization/2026-09-23/evidence/run-pos-postgres-existing.cjs
lifecycle
node what-next/pos-enterprise-modernization/2026-09-23/evidence/run-pos-postgres-existing.cjs
corrections
node what-next/pos-enterprise-modernization/2026-09-23/evidence/run-pos-postgres-existing.cjs full-
commit
node node_modules/eslint/bin/eslint.js services/pos/__tests__/pos-full-commit.postgres.test.ts
```

Each command above returned exit 0. The initial bootstrap runner's commit phase also returned 0; its overall interrupted run did not establish a passing aggregate result. The resume runner JSON files contain exact underlying Jest commands, target and start/completion timestamps. The bootstrap is retained as provenance and should not be rerun just to resume tests.

The existing `run-pos-postgres.cjs` created `stoquify_pos_audit_20260923_d1532fb1` on localhost:5432 by cloning schema only from the local app database; it did not copy application rows. `postgres-target.json` identifies it. The resume runner `run-pos-postgres-existing.cjs` validates this exact retained database, local host and name prefix before running tests. The bootstrap was not rerun. After explicit user approval, a separate runner applied only the existing posting-cycle migration, with its exact SHA checked, to this disposable database. `postgres-approved-posting-cycle.json` and `.sql` retain the execution evidence. The reusable test runner does not apply migrations. No application database rows/schema, live financial operations, broad migrations or production deployment were changed. Generated fixture rows remain in the disposable database for review. The lifecycle test creates and drops only its own isolated schema after a successful CREATE; it refuses an existing schema.

Residual limits and rollout requirements

1. The full real-kernel fixture exposed P2022 schema drift before passing on the repaired disposable schema. **The application database still lacks `ledger_posting_batches.postingCycle`, a high runtime/release blocker.** Automatic review initially rejected incidental test-runner migration; the user then explicitly approved the exact existing migration only for the disposable target. That migration completed with its SHA verified, and both full-commit cases passed. See `full-commit-readiness.md` for exact commands, SQL effects, hash, authorization and outcomes; `pos-schema-column-audit.json` confirms the application remains unchanged. The 21 passing cases do not authorize or establish an application schema rollout, statutory readiness, live provider correctness or all possible financial interleavings.

2. Electronic refunds/voids are deliberately unavailable through this service until a provider reversal/evidence contract is implemented and verified. Do not re-enable them by allowing extra enum values alone. Service-supported refunds/voids and on-account operations are not automatically available in the cashier UI.
3. Receipt hardware/providers, offline capture enrollment UI, cross-cashier handover and partial-item refunds are outside verified delivered paths. Preserve their explicit product limitations in the main workflow map.
4. No financial records should be edited manually to bypass a correction conflict. Support should inspect tenant, terminal, shift, opening drawer, balances and original request/result evidence, then follow the existing reconciliation process.
5. Rollback must not reinstate unsupported electronic correction or unconditional drawer writes. If a deployed regression appears, disable the affected correction entry point while retaining sale/refund/audit evidence; use a reviewed corrective release.

Independent lifecycle fence review

Read-only review of `hooks/posHooks/usePOSCheckoutLifecycle.ts` after the root agent's changes found no blocking defect. Effect cleanup marks the previous closure inactive and advances generation; commands also carry an authority version so a response from before offline/expiry cannot accept a cart after reconnect. The synchronous readiness/command refs prevent two same-tick commands. Refresh response latency is deducted from the lease. When an invalidated request finally returns, readiness remains false until another refresh, which can take the normal 20-second interval; this is a conservative availability tradeoff. Cart invalidation prefixes correctly match query keys that append actor/organization scope. Verification of this hook remains the root agent's regression suite.